BVRIT HYDERABAD College of Engineering for Women

Department of Computer Science & Engineering (AI&ML)

MACHINE LEARNING PROJECT

Project Title: GRID PULSE - Household Electricity Consumption Forecasting and Anomaly Detection Using Machine Learning

Team Name	PulseForge
Class / Section	CSE (AI&ML)
Academic Year	2026 - 2027
Faculty Name	Ms.P. Nikitha,Ms.V. Thanuja
Date of Submission	06-11-2026

Team Members

S.No	Roll Number	Student Name
1	24WH1A6610	G. Shamithri
2	24WH1A6615	K. Sahithi Rithvika
3	24WH1A6620	B. Glory Pranavi
4	24WH1A6629	E. Manusree
5	24WH1A6633	E. Sai Spoorthy
6	24WH1A6637	D. Srija

7	25WH5A6601	K. Hari Hansika
8	25WH5A6606	E.Vidyasree

Project Links

GitHub Repository Link	https://github.com/ESpoorthy/GridPulse/blob/main/scripts/train_models.py

Project Description

Problem Statement

Household electricity consumption varies continuously depending on the time of day, appliance usage and daily activities. Analyzing large amounts of historical electricity consumption data manually is difficult and time-consuming.

It is also challenging to predict future electricity consumption and identify unusual power usage patterns using traditional manual analysis.

Therefore, there is a need for an automated system that can analyze historical electricity consumption data, forecast future electricity usage, detect unusual observations and evaluate dataset quality.

To address this problem, we propose GridPulse, a machine learning-based system that uses different forecasting algorithms to analyze historical electricity readings, predict the next 24 hours of electricity consumption and identify unusual consumption patterns through an interactive dashboard.

Abstract

GridPulse is a machine learning-based project developed to analyze historical household electricity consumption and predict future electricity usage. The main aim of this project is to understand electricity consumption patterns, forecast future power requirements and identify unusual electricity consumption observations.

The system uses the UCI Individual Household Electric Power Consumption dataset, which contains electricity readings collected from a household in Sceaux, France, between 2006 and 2010. The collected data is preprocessed, validated and converted from minute-level readings into hourly mean active power values.

Different machine learning algorithms, including Linear Regression, Random Forest and HistGradientBoostingRegressor, are implemented and compared with Persistence and Seasonal Naive baseline models. The models are evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

The proposed system also performs anomaly detection to identify unusual electricity consumption patterns and evaluates the quality of the dataset. A FastAPI backend and Next.js frontend are integrated to display forecasting results, model performance, anomalies and dataset health through an interactive dashboard.

The experimental results show that HistGradientBoosting achieved a one-step test MAE of 0.3180 kW and a recursive 24-hour test MAE of 0.4427 kW. GridPulse provides a systematic approach to historical household electricity consumption analysis, forecasting and anomaly identification.

Code:

```
"""GridPulse API. Responses are computed from generated local data artifacts."""

from __future__ import annotations
```

Project Description

```
import json
import logging
import os
from datetime import datetime, timezone
from pathlib import Path

import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from backend.app.schemas import (
    AnomalyEvent,
    AnomalyResponse,
    DataHealthResponse,
    ForecastPoint,
    ForecastResponse,
    HealthResponse,
    HistoricalPoint,
    ModelComparisonResponse,
    OverviewResponse,
)
from ml.forecasting import recursive_forecast
from ml.anomaly_detection import detect_seasonal_anomalies

logger = logging.getLogger(__name__)
ROOT = Path(__file__).resolve().parents[2]


def _configured_path(variable: str, default: str) -> Path:
    path = Path(os.getenv(variable, default))
    return path if path.is_absolute() else ROOT / path


HOURLY_PATH = _configured_path(
    "GRIDPULSE_HOURLY_DATA", "data/processed/hourly_power.csv"
)
VALIDATION_PATH = _configured_path(
    "GRIDPULSE_VALIDATION_REPORT", "reports/generated/dataset_validation.json"
)
RESULTS_PATH = _configured_path(
    "GRIDPULSE_MODEL_RESULTS", "reports/generated/real_data_results.json"
)
MODEL_PATH = _configured_path(
    "GRIDPULSE_MODEL_ARTIFACT", "artifacts/selected_model.joblib"
)

app = FastAPI(
    title="GridPulse API",
    description="Historical household power analytics",
    version="1.1.0",
    docs_url="/api/docs",
    redoc_url="/api/redoc",
)
origins = [
    origin.strip()
    for origin in os.getenv(
        "GRIDPULSE_CORS_ORIGINS", "http://localhost:3000,http://127.0.0.1:3000"
    ).split(",")
    if origin.strip()
]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["GET"],
    allow_headers=["*"],
)


def _json(path: Path, label: str) -> dict:
    try:
        return json.loads(path.read_text(encoding="utf-8"))
```

```
    except FileNotFoundError as exc:
        raise HTTPException(
            status_code=503,
            detail=f"{label} is unavailable. Generate it with scripts/train_models.py.",
        ) from exc
    except (OSError, json.JSONDecodeError) as exc:
        logger.exception("Unable to read %s", label)
        raise HTTPException(
            status_code=503, detail=f"{label} is unavailable or malformed."
        ) from exc


def _hourly() -> pd.DataFrame:
    try:
        frame = pd.read_csv(HOURLY_PATH, parse_dates=["timestamp"])
    except FileNotFoundError as exc:
        raise HTTPException(
            status_code=503,
            detail="Validated hourly data is unavailable. Run scripts/train_models.py.",
        ) from exc
    except (OSError, ValueError, pd.errors.ParserError) as exc:
        logger.exception("Unable to read processed hourly data")
        raise HTTPException(
            status_code=503, detail="Validated hourly data is unavailable or malformed."
        ) from exc
    required = {"timestamp", "global_active_power_mean_kw"}
    if not required.issubset(frame.columns):
        raise HTTPException(
            status_code=503, detail="Validated hourly data has an invalid schema."
        )
    frame["global_active_power_mean_kw"] = pd.to_numeric(
        frame["global_active_power_mean_kw"], errors="coerce"
    )
    frame = frame.dropna(
        subset=["timestamp", "global_active_power_mean_kw"]
    ).sort_values("timestamp")
    if frame.empty or frame["timestamp"].duplicated().any():
        raise HTTPException(
            status_code=503,
            detail="Validated hourly data has no usable rows or has duplicate timestamps.",
        )
    return frame.reset_index(drop=True)


def _latest_contiguous_history(frame: pd.DataFrame) -> pd.DataFrame:
    gaps = frame["timestamp"].diff().ne(pd.Timedelta(hours=1))
    starts = frame.index[gaps]
    start = int(starts[-1]) if len(starts) else 0
    return frame.iloc[start:].tail(1000).reset_index(drop=True)


@app.get("/health", response_model=HealthResponse, tags=["System"])
async def health_check():
    return HealthResponse(
        status="healthy",
        version=app.version,
        data_available=HOURLY_PATH.is_file(),
        models_loaded=MODEL_PATH.is_file(),
    )


@app.get(
    "/api/v1/model-metrics", response_model=ModelComparisonResponse, tags=["Models"]
)
async def get_model_metrics():
    result = _json(RESULTS_PATH, "Experiment results")
    if not result.get("models") or not result.get("test_period"):
        raise HTTPException(
            status_code=503, detail="Experiment results are incomplete."
        )
    models = []
    for name, values in result["models"].items():
        metrics = values.get("test")
```

```
        if not metrics:
            continue
        recursive = values.get("recursive_validation", {})
        recursive_metrics = recursive.get("metrics")
        models.append(
            {
                "name": name.replace("_", " ").title(),
                "mae": metrics["mae"],
                "rmse": metrics["rmse"],
                "train_samples": result.get("fit_samples", result["train_samples"]),
                "test_samples": result["test_samples"],
                "training_time": None,
                "is_baseline": name in {"persistence", "seasonal_naive_daily"},
                "recursive_validation": (
                    {
                        **recursive_metrics,
                        "forecast_origins": recursive.get("forecast_origins", 0),
                        "evaluated_hours": recursive.get("evaluated_hours", 0),
                        "evaluation_period": recursive.get("evaluation_period", {}),
                        "step_group_metrics": recursive.get("step_group_metrics", {}),
                    }
                    if recursive_metrics
                    else None
                ),
            }
        )
    if not models:
        raise HTTPException(status_code=503, detail="No test metrics are available.")
    models.sort(key=lambda item: (item["mae"], item["name"]))
    return ModelComparisonResponse(
        models=models,
        best_model=result["best_model"].replace("_", " ").title(),
        evaluation_period=result["test_period"],
        split_ratio=result["split_ratio"],
        forecast_horizon=int(result.get("horizon_hours", 1)),
        selection_metric=result.get("selection_metric", "one-step validation MAE"),
        recursive_test=result.get("recursive_test"),
        is_demo=False,
    )


@app.get(
    "/api/v1/dataset-health", response_model=DataHealthResponse, tags=["Data Quality"]
)
async def get_dataset_health():
    validation, frame = _json(VALIDATION_PATH, "Dataset validation report"), _hourly()
    target = frame["global_active_power_mean_kw"]
    raw_count = int(validation["row_count"])
    missing = int(
        validation.get("missing_numeric_values", {}).get("Global_active_power", 0)
    ) + int(validation.get("numeric_parse_errors", {}).get("Global_active_power", 0))
    bins = int(validation["hourly_aggregation"]["hourly_bins_in_time_range"])
    eligible = int(validation["hourly_aggregation"]["eligible_hours"])
    if bins <= 0:
        raise HTTPException(
            status_code=503,
            detail="Dataset validation report has invalid coverage counts.",
        )
    dist = (
        frame.assign(hour=frame.timestamp.dt.hour)
        .groupby("hour")["global_active_power_mean_kw"]
        .agg(avg_power="mean", sample_count="count")
        .reset_index()
    )
    return DataHealthResponse(
        dataset_name=validation["dataset_name"],
        source_file=validation["input_file"],
        quality={
            "total_rows": raw_count,
            "valid_rows": int(
                validation.get("non_missing_target_rows", raw_count - missing)
            ),
            "missing_rows": missing,
```

Project Description

```
            "missing_percentage": (
                round(missing / raw_count * 100, 3) if raw_count else 0
            ),
            "duplicate_rows_excluded": int(
                validation.get("duplicate_rows_excluded_from_aggregation", 0)
            ),
            "timestamp_parse_failures": int(
                validation.get("timestamp_parse_failures", 0)
            ),
            "missing_minute_timestamps": int(
                validation.get("missing_minute_timestamps_in_range", 0)
            ),
            "excluded_hourly_intervals": int(
                validation["hourly_aggregation"].get(
                    "excluded_hours_below_coverage_threshold", 0
                )
            ),
            "date_range": {
                "start": validation["first_timestamp"],
                "end": validation["last_timestamp"],
            },
            "coverage": {
                "total_hours": bins,
                "eligible_hours": eligible,
                "coverage_percentage": round(eligible / bins * 100, 2),
            },
            "warnings": validation.get("warnings", []),
            "validation_status": validation.get("validation_status", "unknown"),
        },
        statistics={
            "min": float(target.min()),
            "max": float(target.max()),
            "mean": float(target.mean()),
            "median": float(target.median()),
            "std": float(target.std(ddof=1) or 0),
        },
        hourly_distribution=dist.to_dict("records"),
        is_demo=False,
    )


@app.get("/api/v1/overview", response_model=OverviewResponse, tags=["Dashboard"])
async def get_overview():
    frame = _hourly()
    target = "global_active_power_mean_kw"
    end = frame.timestamp.max()
    indexed = frame.set_index("timestamp")[target].sort_index()
    day_start = end.floor("D")
    day = indexed.loc[(indexed.index >= day_start) & (indexed.index <= end)]
    this_week = indexed.loc[
        (indexed.index > end - pd.Timedelta(days=7)) & (indexed.index <= end)
    ]
    prior_week = indexed.loc[
        (indexed.index > end - pd.Timedelta(days=14))
        & (indexed.index <= end - pd.Timedelta(days=7))
    ]
    if day.empty:
        raise HTTPException(
            status_code=503,
            detail="No observations are available in the latest observation day.",
        )
    change = (
        (
            (float(this_week.mean()) - float(prior_week.mean()))
            / float(prior_week.mean())
            * 100
        )
        if not prior_week.empty and prior_week.mean() != 0
        else 0.0
    )
    trend = "up" if change > 1 else "down" if change < -1 else "stable"
    validation = _json(VALIDATION_PATH, "Dataset validation report")
    events = await get_anomalies()
```

```
    expected_hours = int((end.floor("h") - day_start) / pd.Timedelta(hours=1)) + 1
    return OverviewResponse(
        current_power=float(indexed.iloc[-1]),
        daily_average=float(day.mean()),
        weekly_trend=trend,
        weekly_change=round(change, 2),
        daily_observed_hours=int(day.count()),
        daily_expected_hours=expected_hours,
        daily_coverage_percentage=round(int(day.count()) / expected_hours * 100, 1),
        anomaly_count=events.totalCount,
        last_updated=end.isoformat(),
        data_source=validation["dataset_name"],
        recent_consumption=[
            {"timestamp": ts.isoformat(), "power": float(val)}
            for ts, val in indexed.tail(168).items()
        ],
        is_demo=False,
    )


@app.get("/api/v1/forecast", response_model=ForecastResponse, tags=["Forecasting"])
async def get_forecast():
    if not MODEL_PATH.is_file():
        raise HTTPException(
            status_code=503,
            detail="Selected model artifact is unavailable. Run scripts/train_models.py.",
        )
    import joblib

    try:
        artifact = joblib.load(MODEL_PATH)
        model, metadata = artifact["model"], artifact["metadata"]
        frame = _latest_contiguous_history(_hourly())
        forecast = recursive_forecast(model, frame, horizon=24)
    except (OSError, ValueError, KeyError, TypeError) as exc:
        logger.exception("Unable to generate forecast")
        raise HTTPException(
            status_code=503,
            detail="The selected model could not produce a forecast from available data.",
        ) from exc
    historical = frame.tail(168)
    points = [
        ForecastPoint(
            timestamp=row.timestamp.isoformat(), predicted=float(row.predicted)
        )
        for row in forecast.itertuples()
    ]
    history_points = [
        HistoricalPoint(
            timestamp=row.timestamp.isoformat(),
            actual=float(getattr(row, "global_active_power_mean_kw")),
        )
        for row in historical.itertuples()
    ]
    return ForecastResponse(
        model_name=metadata["best_model"].replace("_", " ").title(),
        horizon=24,
        generated_at=datetime.now(timezone.utc).isoformat(),
        historical=history_points,
        forecast=points,
        confidence=None,
        is_demo=False,
    )


@app.get("/api/v1/anomalies", response_model=AnomalyResponse, tags=["Anomalies"])
async def get_anomalies():
    result, frame = _json(RESULTS_PATH, "Experiment results"), _hourly()
    calibration = result.get("anomaly_calibration")
    if not calibration:
        raise HTTPException(
            status_code=503,
            detail="Validation-calibrated anomaly thresholds are unavailable. Run scripts/train_models.py.",
```

Project Description

```
        )
    thresholds = {
        "low": float(calibration["threshold_low_kw"]),
        "medium": float(calibration["threshold_medium_kw"]),
        "high": float(calibration["threshold_high_kw"]),
    }
    series = frame.set_index("timestamp")["global_active_power_mean_kw"].asfreq("h")
    raw_events, counts, start, end = detect_seasonal_anomalies(series, thresholds)
    events = [AnomalyEvent(**event) for event in raw_events]
    return AnomalyResponse(
        events=events,
        total_count=len(events),
        severity_counts=counts,
        detection_method=calibration["method"],
        threshold_info=f"Validation residual thresholds: low {thresholds['low']:.3f} kW; medium
            {thresholds['medium']:.3f} kW; high {thresholds['high']:.3f} kW.",
        period={"start": start.isoformat(), "end": end.isoformat()},
        is_demo=False,
    )


@app.exception_handler(404)
async def not_found_handler(request, exc):
    return JSONResponse(
        status_code=404,
        content={
            "error": "Not Found",
            "detail": "The requested endpoint does not exist",
            "status": 404,
        },
    )


@app.get("/", tags=["System"])
async def root():
    return {
        "name": "GridPulse API",
        "version": app.version,
        "status": "operational",
        "docs": "/api/docs",
        "health": "/health",
    }
```

Output Screenshots

Fig 1: Overview Dashboard

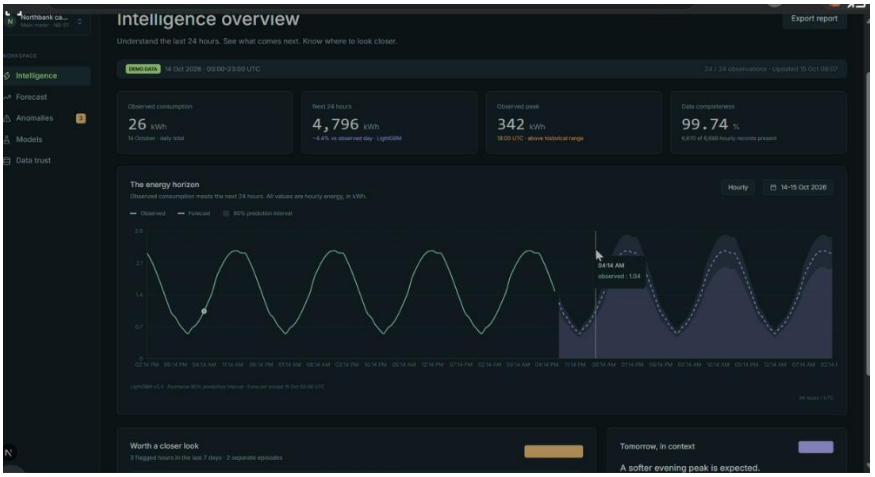
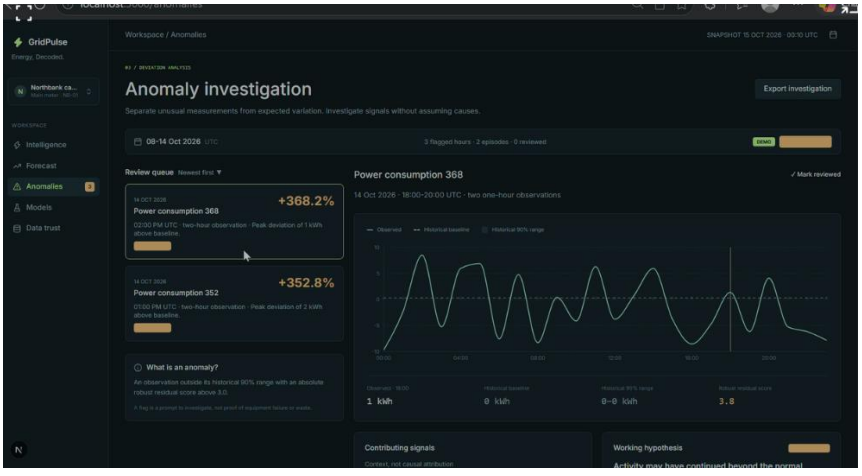


Fig 2: Forecast



Fig 3: Model Comparison



Output Screenshots

Fig 4: Anomaly Detection

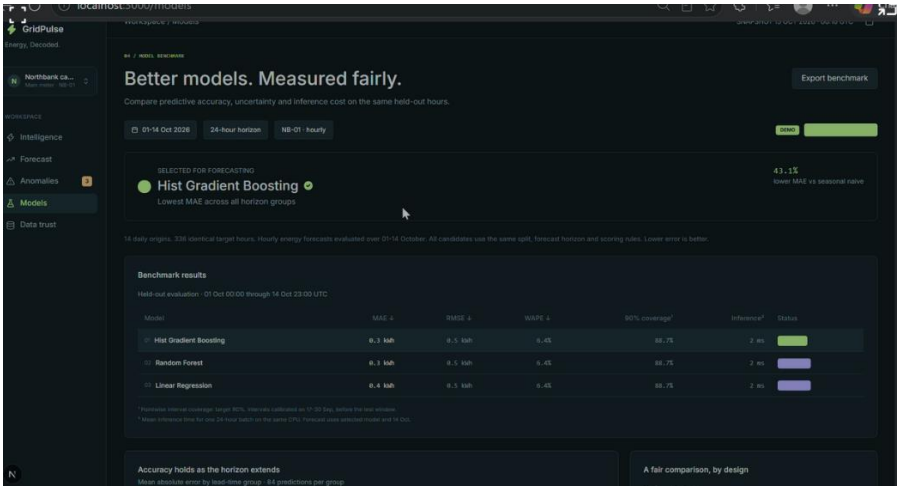


Fig 5: Dataset Health

